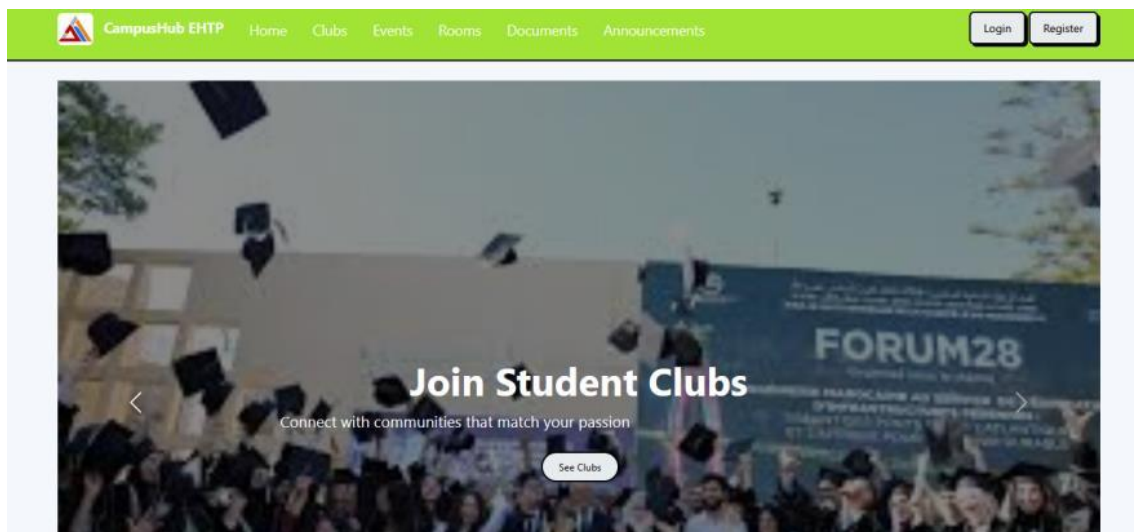


CampusHub

Rapport de Projet Final

Plateforme de Gestion Communautaire et Événementielle Universitaire



Réalisé par : Siham Bouzagar

Technologies : ASP.NET Core MVC & Web API, Entity Framework Core, SQL Server, ASP.NET Core Identity, Bootstrap, Docker

Le : Juin 2026

Table des matières

Table des matières	2
I. Introduction	3
II. Architecture de l'application.....	3
III. Modélisation de la base de données	4
Relations entre entités.....	5
Validation des données.....	5
IV. Sécurité et gestion des rôles.....	6
V. API REST et DTOs	6
DTOs (Data Transfer Objects)	7
Consommation des API.....	7
VI. Déploiement	7
VII. Conformité aux exigences fonctionnelles.....	7
VIII. Captures d'écran de l'application	8
8.1 Authentification et gestion du profil.....	8
8.2 Tableaux de bord	9
8.3 Gestion des clubs	10
8.4 Gestion des événements.....	11
8.5 Réservation de salles.....	12
8.6 Annonces.....	12
8.7 Partage de documents	13
IX. Défis rencontrés et solutions	13
X. Conclusion et perspectives	14

I. Introduction

Les universités font souvent face à des difficultés pour centraliser les activités étudiantes : gestion des clubs, organisation des événements, réservation de salles, diffusion d'annonces et partage de documents entre étudiants, enseignants et administrateurs. CampusHub est une plateforme web moderne conçue pour répondre à ces besoins en offrant un système communautaire universitaire centralisé.

Conformément au cahier des charges du projet, la plateforme prend en charge les modules suivants :

- Gestion des clubs étudiants (création, adhésion, gestion par les administrateurs)
- Organisation et gestion des événements universitaires (hackathons, conférences, ateliers, réunions de clubs)
- Gestion des participations aux événements, avec limite de capacité
- Réservation de salles avec validation des conflits d'horaires
- Diffusion et recherche d'annonces
- Partage de documents (programmes, comptes-rendus, supports de cours)
- Tableaux de bord avec statistiques et indicateurs

L'application a été développée selon les exigences techniques imposées, en utilisant la pile technologique suivante :

- ASP.NET Core MVC pour l'interface utilisateur
- ASP.NET Core Web API pour l'exposition de services REST
- Entity Framework Core comme ORM, avec SQL Server comme système de gestion de base de données
- ASP.NET Core Identity pour l'authentification et la gestion des rôles
- Bootstrap pour une interface responsive et moderne
- HttpClient pour la consommation des API depuis la couche MVC
- Docker pour la conteneurisation et le déploiement

Ce rapport présente l'architecture mise en place, le modèle de données, les mécanismes de sécurité, les API développées, le processus de déploiement, la conformité aux exigences fonctionnelles, ainsi qu'une illustration de l'interface utilisateur à travers des captures d'écran. Il se termine par un retour d'expérience sur les principaux défis rencontrés et les perspectives d'évolution du projet.

II. Architecture de l'application

L'architecture de CampusHub suit un modèle en couches (Layered Architecture) adapté à ASP.NET Core, séparant clairement les responsabilités de présentation, de logique applicative, de domaine et d'accès aux données :

- Couche de présentation : ASP.NET Core MVC pour l'interface utilisateur (Views et Controllers) et Web API pour les points d'accès REST.
- Couche applicative : services tels que EventApiService, ClubApiService et AnnouncementApiService qui consomment les API via HttpClient et orchestrent la logique métier.

- Couche domaine : entités métier, DTOs (EventCreateDto, EventReadDto, etc.) et règles de validation.
- Couche infrastructure : Entity Framework Core pour l'accès aux données, ASP.NET Core Identity pour l'authentification, et gestion des fichiers téléversés (wwwroot/uploads).
- Base de données : SQL Server, avec gestion du schéma via les migrations EF Core.

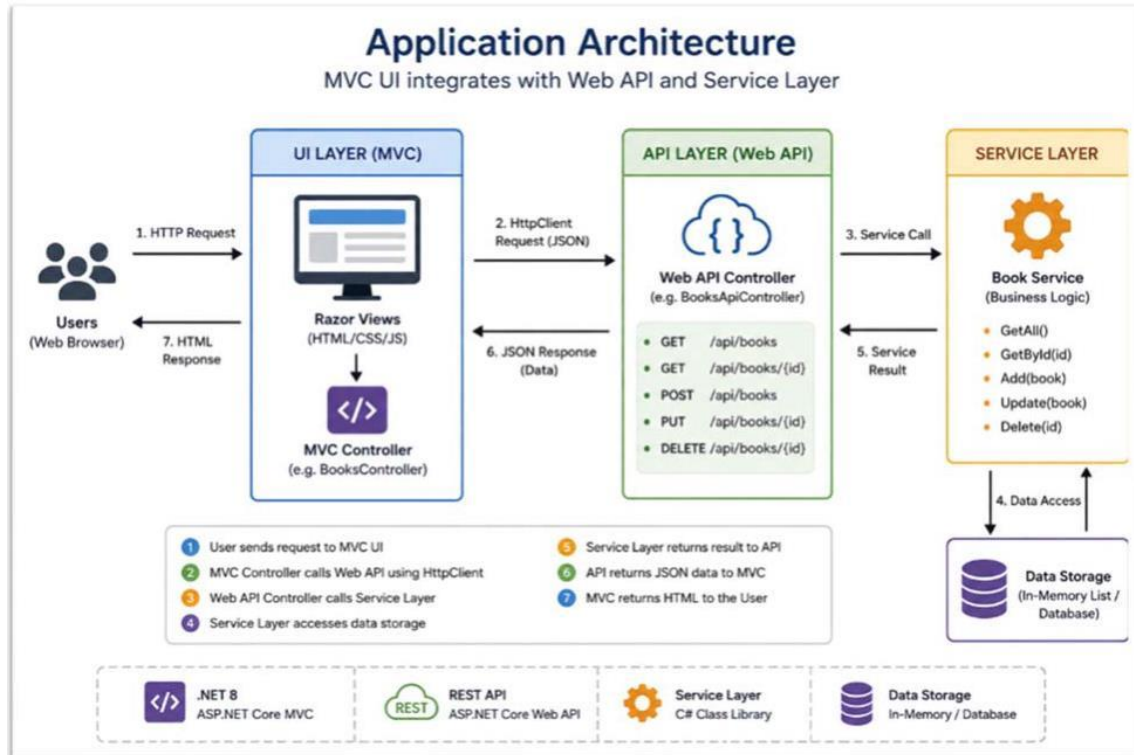


Figure 1 — Architecture générale de l'application : interaction entre la couche MVC, la Web API et la couche de services.

Le flux d'exécution typique d'une requête est le suivant :

1. L'utilisateur interagit avec les vues Razor (MVC).
2. Le contrôleur MVC appelle la Web API via HttpClient (requête JSON).
3. Le contrôleur de la Web API délègue le traitement à la couche de services (logique métier).
4. La couche de services accède aux données via Entity Framework Core.
5. Le résultat est renvoyé à l'API, qui le retourne au format JSON.
6. Le contrôleur MVC reçoit la réponse JSON et construit la vue HTML.
7. La réponse HTML finale est affichée dans le navigateur de l'utilisateur.

Cette séparation des responsabilités facilite la maintenance, les tests et l'évolution indépendante de chaque couche, tout en respectant les exigences techniques du projet (MVC + Web API + EF Core + Identity).

III. Modélisation de la base de données

Conformément aux exigences, une base de données relationnelle a été conçue avec Entity Framework Core et SQL Server. Le tableau ci-dessous résume les entités principales et leurs champs :

Entité	Champs principaux	Description
Club	Id, Name, Description, LogoUrl, CreatedAt	Informations sur les clubs étudiants
Event	Id, Title, Description, EventDate, Location, MaximumParticipants, ImageUrl	Événements organisés (hackathons, conférences, ateliers...)
Announcement	Id, Title, Content, CreatedAt, UserId	Annonces publiées par les enseignants/administrateurs
RoomReservation	Id, RoomName, ReservationDate, StartTime, EndTime, Status, UserId	Demandes de réservation de salles
ClubMembership	Id, ClubId, UserId, JoinedAt	Adhésion d'un étudiant à un club
EventParticipation	Id, EventId, UserId, RegisteredAt	Inscription d'un utilisateur à un événement
ApplicationUser	Id, UserName, Email, FullName	Utilisateur de l'application (extension d'Identity)

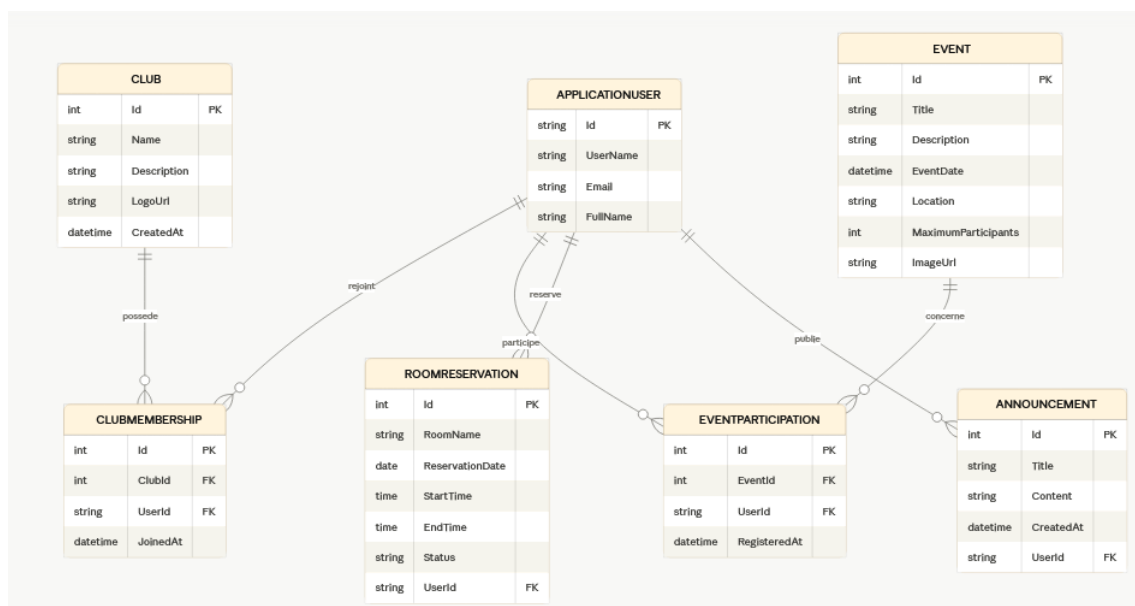


Figure 2 — Schéma relationnel (diagramme entité-association) de la base de données CampusHub.

Relations entre entités

- One-to-Many : Club → ClubMembership (un club possède plusieurs membres)
- One-to-Many : Event → EventParticipation (un événement possède plusieurs participants)
- One-to-Many : ApplicationUser → RoomReservation (un utilisateur effectue plusieurs réservations)
- One-to-Many : ApplicationUser → Announcement (un utilisateur publie plusieurs annonces)

Validation des données

La validation des données est assurée par des Data Annotations sur les entités et les DTOs, notamment :

- [Required] pour les champs obligatoires (titres, noms, dates...)
- [StringLength] pour limiter la taille des champs texte
- [Range] pour contraindre des valeurs numériques (ex. MaximumParticipants)
- [EmailAddress] pour valider le format des adresses e-mail

Les messages de validation correspondants sont affichés directement dans l'interface utilisateur (cf. Figure « Inscription » dans la section Captures d'écran), et le schéma relationnel garantit l'intégrité référentielle (clés étrangères, contraintes d'unicité pour éviter les inscriptions ou adhésions en double).

IV. Sécurité et gestion des rôles

La sécurité de l'application repose sur ASP.NET Core Identity, avec un contrôle d'accès basé sur les rôles (Role-Based Access Control). Trois rôles principaux sont définis :

Rôle	Responsabilités
Admin	Gestion complète du système : création/édition/suppression des clubs et événements, gestion des utilisateurs, approbation ou rejet des réservations de salles, supervision du dashboard global.
Teacher	Supervision des événements, publication d'annonces, demandes de réservation de salles, partage de documents.
Student	Consultation et adhésion aux clubs, inscription/annulation aux événements, demandes de réservation, consultation des annonces et documents.

Les fonctionnalités d'authentification et de gestion des utilisateurs implémentées comprennent :

- Inscription et connexion/déconnexion des utilisateurs
- Validation des mots de passe (règles de complexité Identity)
- Autorisation basée sur les rôles (attributs [Authorize(Roles="...")])
- Téléversement et modification de la photo de profil
- Édition du profil utilisateur (nom complet, e-mail)

Au niveau de l'application, les mécanismes suivants protègent les ressources sensibles :

- Authentification obligatoire pour accéder aux fonctionnalités personnalisées (profil, réservations, participations)
- Autorisation différenciée selon le rôle de l'utilisateur connecté
- Protection des contrôleurs et actions sensibles (gestion des clubs, événements, validation des réservations)
- Sécurisation des pages d'administration, accessibles uniquement au rôle Admin

V. API REST et DTOs

Conformément aux exigences, des API RESTful ont été développées avec ASP.NET Core Web API. L'API minimale requise pour la gestion des événements expose les points de terminaison suivants :

Méthode	Endpoint	Description
GET	/api/events	Récupérer la liste de tous les événements

Méthode	Endpoint	Description
GET	/api/events/{id}	Récupérer le détail d'un événement
POST	/api/events	Créer un nouvel événement
PUT	/api/events/{id}	Modifier un événement existant
DELETE	/api/events/{id}	Supprimer un événement

Des API complémentaires ont également été mises en place pour les clubs, les annonces et les réservations, suivant le même schéma REST (GET / POST / PUT / DELETE).

DTOs (Data Transfer Objects)

La communication entre la couche MVC et la Web API repose exclusivement sur des DTOs, afin de découpler les entités de la base de données du contrat d'API. Les principaux DTOs définis sont :

- EventCreateDto : champs nécessaires à la création d'un événement
- EventReadDto : représentation d'un événement renvoyée par l'API
- ClubDto : représentation d'un club (création / lecture)
- AnnouncementDto : représentation d'une annonce

Consommation des API

L'application MVC consomme les API REST via HttpClient, à travers des classes de service dédiées qui encapsulent les appels HTTP et la désérialisation JSON :

- EventApiService
- ClubApiService
- AnnouncementApiService

VI. Déploiement

L'application a été conçue pour fonctionner dans des conteneurs Docker, conformément aux exigences de déploiement :

- Un Dockerfile permet de construire l'image de l'application ASP.NET Core.
- Un fichier docker-compose.yml orchestre le conteneur applicatif et le conteneur SQL Server, en définissant les variables d'environnement nécessaires (chaîne de connexion, mots de passe, ports).
- Un reverse proxy IIS est utilisé pour exposer l'application déployée, conformément aux exigences de déploiement du cahier des charges.

Cette approche permet de garantir la portabilité de l'application et de faciliter sa mise en production sur différents environnements.

VII. Conformité aux exigences fonctionnelles

Le tableau suivant synthétise la couverture des modules fonctionnels exigés par le cahier des charges et leur état d'implémentation dans CampusHub :

Module	Exigences clés	Statut
Authentification & utilisateurs	Inscription, connexion/déconnexion, validation du mot de passe, rôles, upload et édition du profil	Implémenté
Gestion des clubs	Consultation, adhésion et retrait (étudiants) ; création, édition et suppression (administrateurs)	Implémenté
Gestion des événements	Création, édition, suppression et gestion des participants (enseignants/admin) ; consultation, inscription et annulation (étudiants)	Implémenté
Participation aux événements	Prévention des inscriptions en double, limite de capacité, affichage du nombre de participants	Implémenté
Annonces	Publication par enseignants/admin ; consultation et recherche par les étudiants	Implémenté
Réservation de salles	Demande par étudiants/enseignants, approbation ou rejet par l'admin, prévention des conflits d'horaires	Implémenté
Partage de documents	Upload, téléchargement et validation des fichiers (programmes, comptes-rendus, supports)	Implémenté
Dashboard	Statistiques (clubs, étudiants, événements, réservations), événements à venir, indicateurs de participation	Implémenté
Upload de fichiers	Logos de clubs, images d'événements et documents stockés dans wwwroot/uploads	Implémenté
Recherche et filtrage	Recherche d'événements et de clubs, filtrage par catégorie et par date	Implémenté

VIII. Captures d'écran de l'application

Cette section illustre les principales fonctionnalités de CampusHub à travers des captures d'écran de l'application, organisées par module fonctionnel.

8.1 Authentification et gestion du profil

Le formulaire d'inscription affiche des messages de validation en temps réel (champs obligatoires) :

The screenshot shows a registration form with the following fields and messages:

- Full Name**: A text input field with a red error message below it: "The FullName field is required."
- Email**: A text input field containing "admin@campushub.com" with a red error message below it: "The Email field is required."
- Password**: A text input field with masked characters (dots) and a red error message below it: "The Password field is required."
- Confirm Password**: A text input field.
- Student**: A text input field.
- Register**: A blue button at the bottom of the form.

Figure 3 — Formulaire d'inscription avec messages de validation.

Chaque utilisateur dispose d'une page de profil lui permettant de modifier ses informations et de téléverser sa photo :

[Home](#)
[Clubs](#)
[Events](#)
[Rooms](#)
[Documents](#)
[Annoncements](#)

Super Admin

My Profile

Full Name

Email

Parcourir...

Aucun fichier sélectionné.

Save Changes

My Profile

Full Name

Email

Parcourir...

Aucun fichier sélectionné.

Save Changes

Profil de l'administrateur
Profil d'un enseignant

8.2 Tableaux de bord

Le tableau de bord administrateur présente une vue d'ensemble des indicateurs clés (clubs, étudiants, événements, réservations), les événements à venir et les statistiques de participation :



Figure 4 — Tableau de bord de l'administrateur.

Le tableau de bord enseignant propose une vue personnalisée centrée sur ses événements, ses réservations et ses documents récents :

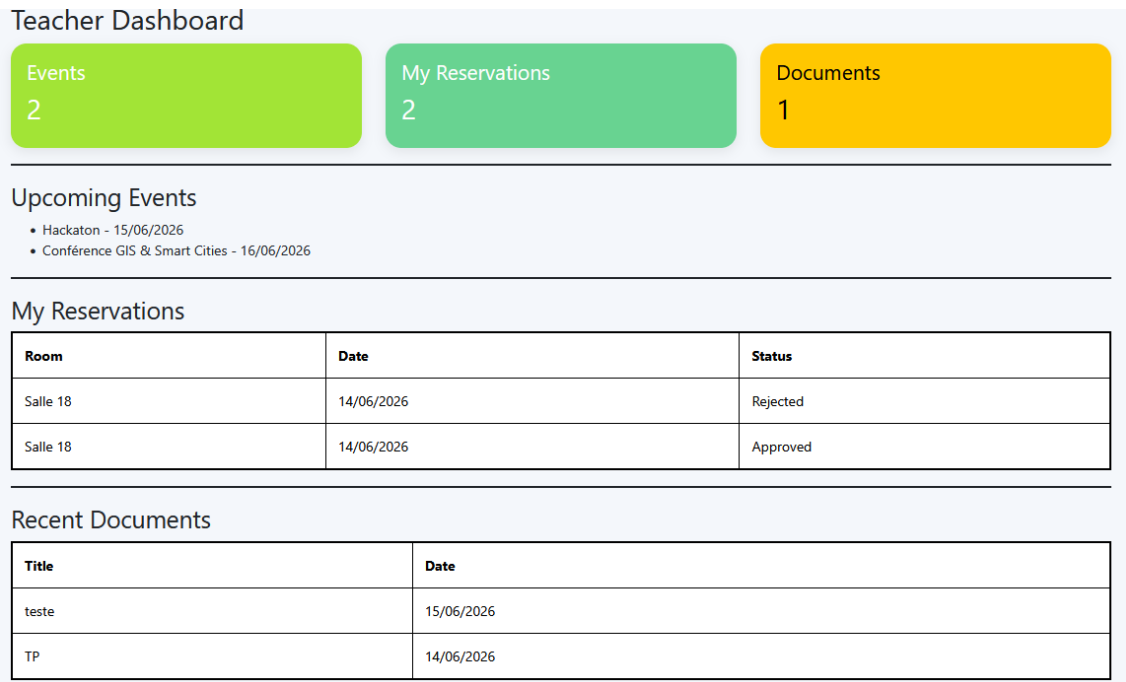


Figure 5 — Tableau de bord de l'enseignant.

8.3 Gestion des clubs

La liste des clubs propose une recherche et un filtrage, ainsi que des actions de gestion réservées aux administrateurs (Edit / Delete) :

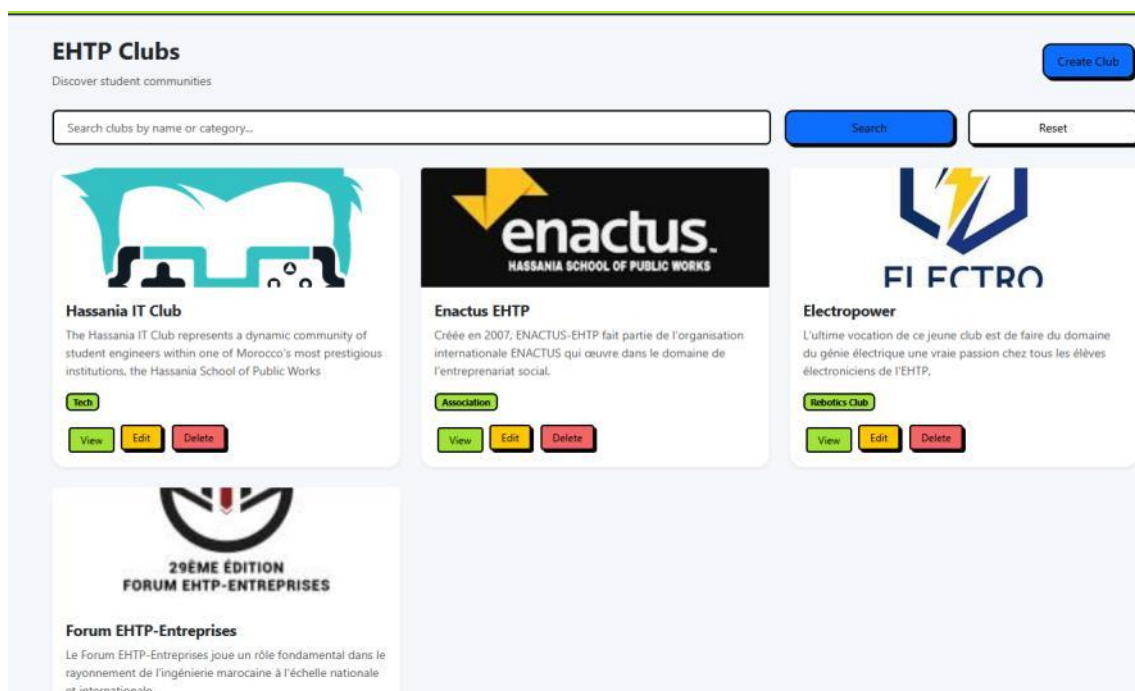


Figure 6 — Liste des clubs avec recherche et actions d'administration.

Un étudiant peut consulter le détail d'un club, le rejoindre puis le quitter ; l'interface met à jour le statut d'adhésion et affiche un message de confirmation :

Vous avez rejoint le club avec succès.  Enactus EHTP Association Créée en 2007, ENACTUS-EHTP fait partie de l'organisation internationale ENACTUS qui œuvre dans le domaine de l'entrepreneuriat social. Créé le: 29/05/2026 Join Club Leave Club Back	Vous avez quitté le club.  Enactus EHTP Association Créée en 2007, ENACTUS-EHTP fait partie de l'organisation internationale ENACTUS qui œuvre dans le domaine de l'entrepreneuriat social. Créé le: 29/05/2026 Join Club Back
<i>Adhésion confirmée au club</i>	<i>Retrait confirmé du club</i>

8.4 Gestion des événements

Les enseignants et administrateurs peuvent créer et modifier les événements (titre, description, lieu, date, capacité maximale, image) :

Edit Event

Title

Description

Location

Date

MaximumParticipants

Current image:



Change Image

Figure 7 — Formulaire de modification d'un événement.

La liste des événements à venir propose une recherche par titre ainsi qu'un filtrage par catégorie et par date :

Upcoming Events

Search

Category

Date

Figure 8 — Recherche et filtrage des événements.

Un étudiant peut s'inscrire à un événement puis annuler sa participation ; le compteur de participants (X / capacité maximale) est mis à jour automatiquement, et une inscription en double est empêchée :

Registered successfully!  Conférence GIS & Smart Cities Ehtp Casablanca Date: 16 juin 2026 14:30 Participants: 17/20 International Conference on GIS for Urban Planning and Smart Cities is a globally recognized research and development association. Register Cancel Participation Back	Participation cancelled.  Conférence GIS & Smart Cities Ehtp Casablanca Date: 16 juin 2026 14:30 Participants: 0/20 International Conference on GIS for Urban Planning and Smart Cities is a globally recognized research and development association. Register Back
<i>Inscription confirmée (1/20 participants)</i>	<i>Participation annulée (0/20 participants)</i>

8.5 Réservation de salles

Les administrateurs disposent d'une vue listant l'ensemble des demandes de réservation avec leur statut (Approved / Rejected / Pending) :

Room Reservations					
Room	Date	Start	End	Status	Actions
Salle 18	14/06/2026	08:30:00	10:30:00	Approved	No actions
12	20/06/2026	12:30:00	13:45:00	Rejected	No actions
Salle 18	14/06/2026	08:30:00	10:30:00	Approved	No actions
Salle 18	14/06/2026	08:30:00	10:30:00	Rejected	No actions

Figure 9 — Vue administrateur des réservations de salles.

Chaque utilisateur (étudiant ou enseignant) peut suivre l'état de ses propres demandes de réservation :

CampusHub EHTP				Home	Clubs	Events	Rooms	Documents	Announcements	Assil
My Reservations										
Salle 18			14/06/2026 00:00:00							Approved
Salle 18			14/06/2026 00:00:00							Rejected
Salle de conference			20/06/2026 00:00:00							Pending

Figure 10 — Suivi des réservations personnelles d'un utilisateur.

8.6 Annonces

Les enseignants et administrateurs peuvent publier des annonces ; les étudiants peuvent les consulter et les rechercher :



Figure 11 — Module Annonces : publication, recherche et confirmation de publication.

8.7 Partage de documents

Le module de partage de documents permet à chaque rôle de téléverser des fichiers (cours, comptes-rendus, supports) et de les télécharger :

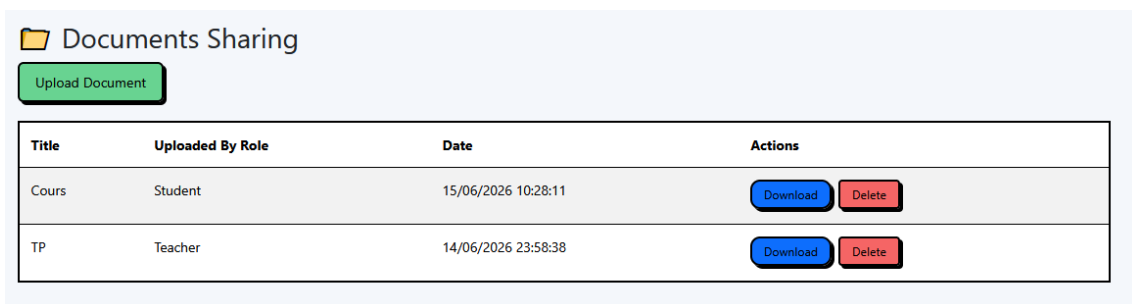


Figure 12 — Liste des documents partagés, avec actions de téléchargement et de suppression.

8.9. Deployment Requirements

☐	▼	☐	campushub	-	-	-
☐		☐	db	586940cbb482	mssql/serv	1433:1433
☐		☐	app	d49e0035c513	campushul	5000:8080

IX. Défis rencontrés et solutions

Le développement de CampusHub a permis de mettre en pratique plusieurs concepts avancés d'ASP.NET Core, mais a également présenté un certain nombre de défis techniques, résumés dans le tableau suivant :

Défi rencontré	Solution apportée
Configuration des rôles avec ASP.NET Core Identity et protection des routes selon le rôle de l'utilisateur.	Mise en place d'un système de seed des rôles (Admin, Teacher, Student) au démarrage de l'application et utilisation d'attributs <code>[Authorize(Roles="...")]</code> sur les contrôleurs et actions sensibles.

Défi rencontré	Solution apportée
Prévention des conflits de réservation de salles (chevauchement d'horaires pour une même salle et une même date).	Ajout d'une logique de validation côté serveur comparant les plages StartTime/EndTime des réservations existantes avant d'enregistrer une nouvelle demande, en complément des Data Annotations.
Mise en place de Docker et communication entre les conteneurs applicatif et SQL Server.	Création d'un Dockerfile et d'un docker-compose.yml, adaptation des chaînes de connexion et configuration du réseau entre conteneurs.
Difficultés de connexion entre les conteneurs Docker et SQL Server.	Plusieurs erreurs de résolution d'hôte et de connexion ont été rencontrées lors de l'utilisation des noms de services Docker (campushub-db, campushub-app). Des ajustements de configuration ont été effectués sur les chaînes de connexion, les ports et les variables d'environnement.
Déploiement complet de l'application avec Docker.	Bien que les images Docker de l'application et de la base de données aient été construites avec succès, le déploiement final n'a pas pu être finalisé en raison de problèmes persistants de communication entre les services et de configuration de l'environnement d'exécution. Les fonctionnalités ont néanmoins été validées en mode développement local via dotnet run.
Conteneurisation de l'application avec Docker et configuration de la connexion à SQL Server entre conteneurs.	Rédaction d'un Dockerfile pour l'application ASP.NET Core et d'un docker-compose.yml orchestrant les conteneurs applicatif et base de données, avec gestion des chaînes de connexion via variables d'environnement.

X. Conclusion et perspectives

CampusHub répond aux exigences fonctionnelles, techniques et de sécurité définies dans le cahier des charges. La plateforme offre une expérience utilisateur intuitive — gestion des clubs, des événements, des réservations, des annonces et des documents — tout en s'appuyant sur une architecture robuste, modulaire et conforme aux bonnes pratiques ASP.NET Core (MVC, Web API, EF Core, Identity, DTOs, services HttpClient).

Ce projet a permis de consolider des compétences clés en développement full-stack .NET, notamment la mise en place d'une architecture en couches, la conception d'un modèle de données relationnel, la sécurisation d'une application via Identity et les rôles, la création et la consommation d'API REST, ainsi que la conteneurisation avec Docker.

Plusieurs perspectives d'évolution pourraient enrichir la plateforme :

- Intégration de notifications par e-mail (par exemple lors de la validation d'une réservation ou de la publication d'une annonce)
- Ajout de statistiques avancées et de graphiques interactifs sur le dashboard (Chart.js)
- Mise en place d'un pipeline de déploiement automatisé (CI/CD) pour le conteneur Docker
- Extension du module de recherche avec des filtres combinés (catégorie, date, statut) sur l'ensemble des modules

Dans son ensemble, CampusHub constitue une base solide et extensible pour des applications communautaires universitaires réelles.